

Robo Rescue

Rapport Technique

Lionel DOFONHAKOU

7 juin 2026

Table des matières

1	Introduction	2
2	Aperçu du jeu	2
3	Architecture générale du projet	3
4	Structures de données	3
4.1	Position	3
4.2	Carte	3
4.3	Joueur	3
4.4	Jeu	4
5	Gestion graphique avec SDL	4
5.1	Principe général	4
5.2	SDL_Window	4
5.3	SDL_Renderer	4
5.4	SDL_Texture	5
6	Cycle de rendu SDL	5
6.1	Étape 1 : Effacer l'écran	5
6.2	Étape 2 : Dessiner les éléments	5
6.3	Étape 3 : Afficher le résultat	5
7	Gestion des événements	5
8	Affichage de texte	6
9	Musique et effets sonores	6
10	Mémo SDL pour les futurs projets	6
11	Conclusion	7

1 Introduction

Robo Rescue est un jeu de type labyrinthe développé en langage C à l'aide de la bibliothèque SDL2.

Le joueur contrôle un robot qui doit atteindre une étoile représentant la sortie du niveau tout en gérant plusieurs ressources limitées.

L'objectif principal de ce projet n'était pas uniquement de produire un jeu fonctionnel mais également de se familiariser avec le développement graphique en C à l'aide de SDL2 et de consolider plusieurs notions fondamentales étudiées durant les enseignements de programmation.

Parmi les notions révisées figurent notamment :

- la programmation modulaire ;
- les structures de données ;
- les énumérations ;
- les pointeurs ;
- l'allocation dynamique ;
- la gestion mémoire ;
- les Makefiles ;
- le débogage avec Valgrind.

2 Aperçu du jeu

La Figure 1 présente une capture d'écran de l'interface graphique du jeu.



FIGURE 1 – Capture d'écran de Robo Rescue

L'interface est composée de plusieurs éléments :

- la carte du niveau ;
- le joueur représenté par un robot ;
- les différents types de terrain ;
- l'étoile indiquant la sortie ;
- l'affichage des ressources du joueur ;
- les messages de victoire ou d'échec.

3 Architecture générale du projet

Le projet est organisé en plusieurs modules indépendants.

- `map.c` / `map.h` : gestion des cartes.
- `player.c` / `player.h` : gestion du joueur.
- `game.c` / `game.h` : logique du jeu.
- `render.c` / `render.h` : affichage SDL.
- `main.c` : boucle principale.

Cette organisation permet de séparer les responsabilités et d'obtenir un code plus lisible et maintenable.

4 Structures de données

4.1 Position

Toutes les positions du jeu sont représentées par :

```
struct location_t {  
    int x;  
    int y;  
};
```

Cette structure est utilisée pour :

- la position du joueur ;
- la position de sortie ;
- les futurs objets mobiles.

4.2 Carte

La carte est stockée sous forme de matrice dynamique.

```
struct map_t {  
    enum ground_t **grid;  
    int width;  
    int height;  
};
```

Chaque case contient un type de terrain :

- GRASS
- FLOWER
- FIRE
- SAND
- WALL

Le chargement est effectué à partir d'un fichier texte contenant les dimensions puis les valeurs de la grille.

4.3 Joueur

```
struct player_t {  
    struct location_t player_pos;  
    int resources[NB_RESOURCES];  
};
```

Le joueur possède :

- une position ;

- des points de déplacement ;
- une réserve d'eau ;
- un temps restant.

4.4 Jeu

```
struct game_t {
    struct map_t map;
    struct player_t *player;
    struct location_t exit_location;
    struct location_t start_location;
};
```

Cette structure représente l'état complet d'une partie.
Elle constitue le lien entre tous les autres composants du programme.

5 Gestion graphique avec SDL

5.1 Principe général

Pour construire une application SDL, il faut retenir trois objets fondamentaux :

1. `SDL_Window`
2. `SDL_Renderer`
3. `SDL_Texture`

La compréhension de ces trois concepts suffit à reconstruire la majorité des projets SDL.

5.2 `SDL_Window`

Une fenêtre représente la zone visible de l'application.
Création :

`SDL_CreateWindow ( . . . )`

Rôle :

- définir la taille de la fenêtre ;
- définir son titre ;
- permettre le redimensionnement.

5.3 `SDL_Renderer`

Le renderer est l'objet le plus important de SDL.

On peut le voir comme le moteur de dessin de l'application.

Toutes les opérations graphiques transitent par lui.

Création :

`SDL_CreateRenderer ( . . . )`

Fonctions principales :

- effacer l'écran ;
- dessiner des images ;
- dessiner des lignes ;
- dessiner des rectangles ;
- afficher le résultat final.

5.4 SDL_Texture

Une texture correspond à une image stockée en mémoire graphique.

Dans le projet :

- player.png
- grass.png
- wall.png
- flower.png
- fire.png
- sand.png
- star.png

sont converties en textures grâce à :

`IMG_Load ( ... )`

`SDL_CreateTextureFromSurface ( ... )`

6 Cycle de rendu SDL

Chaque image affichée à l'écran suit le même cycle.

6.1 Étape 1 : Effacer l'écran

`SDL_RenderClear ( renderer ) ;`

Permet de supprimer l'image précédente.

6.2 Étape 2 : Dessiner les éléments

Toutes les textures sont dessinées avec :

`SDL_RenderCopy ( ... )`

Cette fonction est probablement la plus utilisée dans tout projet SDL.

6.3 Étape 3 : Afficher le résultat

`SDL_RenderPresent ( renderer ) ;`

Le contenu du renderer devient visible.

7 Gestion des événements

SDL utilise une file d'événements.

La récupération des événements se fait grâce à :

`SDL_PollEvent ( ... )`

Dans Robo Rescue, les événements utilisés sont :

- flèche haut ;
- flèche bas ;
- flèche gauche ;
- flèche droite ;
- fermeture de la fenêtre.

8 Affichage de texte

SDL ne gère pas directement les polices.

L'extension `SDL_ttf` est nécessaire.

Étapes :

1. Charger la police.
2. Générer une surface texte.
3. Transformer la surface en texture.
4. Afficher la texture.

Fonctions à retenir :

```
TTF_OpenFont ( ... )  
TTF_RenderText_Solid ( ... )  
SDL_CreateTextureFromSurface ( ... )  
SDL_RenderCopy ( ... )
```

9 Musique et effets sonores

`SDL_mixer` permet de gérer l'audio.

Fonctions importantes :

```
Mix_OpenAudio ( ... )  
Mix_LoadMUS ( ... )  
Mix_PlayMusic ( ... )  
Mix_FreeMusic ( ... )
```

Ces fonctions permettent d'ajouter une musique de fond ainsi que des effets sonores.

10 Mémo SDL pour les futurs projets

Si je devais recommencer un projet SDL dans plusieurs années, la procédure à suivre serait :

1. Initialiser SDL avec `SDL_Init()`.
2. Créer une fenêtre avec `SDL_CreateWindow()`.
3. Créer un renderer avec `SDL_CreateRenderer()`.
4. Charger les textures avec `SDL_image`.
5. Mettre en place une boucle principale.
6. Récupérer les événements avec `SDL_PollEvent()`.
7. Dessiner les textures avec `SDL_RenderCopy()`.
8. Afficher le résultat avec `SDL_RenderPresent()`.
9. Libérer toutes les ressources à la fermeture.

Cette séquence constitue le squelette de quasiment toutes les applications SDL2.

11 Conclusion

Ce projet a permis de découvrir concrètement le développement graphique en langage C avec SDL2.

Au-delà de l'aspect ludique, il a servi de support pour réviser de nombreuses notions fondamentales de programmation tout en développant une meilleure compréhension de l'organisation d'un projet logiciel complet.

Les connaissances acquises durant ce projet constituent une base solide pour le développement de futures applications graphiques ou de jeux vidéo plus complexes.